

**Программа автоматического составления
школьного расписания (проект АМСУР)**

Проект на образовательную смену

УО «Национальный детский технопарк»

Направление: «Информационные и компьютерные технологии»

Автор: Крупко Кирилл Николаевич, учащийся 11 «Б» класса

ГУО «Средняя школа № 8 г. Жодино»

Научный руководитель: Бадеева Е. В.

Жодино, 2026

Оглавление

Введение	2
Основная часть. Возможности программы	3
Основная часть. Ядро и архитектура	4
Основная часть. Замеры	4
Основная часть. Нормы и ограничения	5
Заключение	6
Список использованных источников	6

Введение

Завуч в начале сентября собирает расписание несколько дней. Я видел, как это выглядит изнутри, там сотни условий сразу. Учитель не может быть в двух кабинетах одновременно, у детей не должно быть дыр посреди дня, у всех есть свои смены и подгруппы, и еще куча разных условий. Вручную это может быть очень долго, и окна все равно лезут.

Я решил проверить, а смогу ли я, используя современные технологии (ИИ) составить расписание качественнее и быстрее, чем человек, который занимается этим в школах. Не буду долго тянуть — НЕТ, так сделать не выйдет даже при современных технологиях вроде нейросетей. Их проблема заключается в том, что они все же продолжают галлюцинировать, выдавать порой не точную информацию, стараться схитрить, дабы отчитаться пользователю о том, что они все сделали. Также не стоит забывать о том, что через некоторое время, когда контекстное окно начнет забиваться, то она может забывать часть требований (например «чередование сложных и легких уроков»). После того как я проверил и убедился в том, что не все так просто, мне пришла в голову идея о создании программы, при помощи которой можно это посчитать автоматом на обычном школьном компьютере. Сразу скажу честно, идеального расписания за короткое время не бывает. Задача относится к NP трудным (NP-трудная задача считается сверхсложной: для нее компьютер не может быстро найти точный ответ, а правильность чужого решения невозможно быстро проверить.), вариантов слишком много. Поэтому все нормальные программы ищут лучшее найденное за отведенное

время, моя тоже. Эта фраза даже в окне программы висит, я ее не прячу.

Что есть готового, мне не подошло. Платные конструкторы стоят денег, они закрытые, там нужен специалист. Бесплатный конструктор FET [2] считает минутами, он на английском и выдает один вариант без пояснений. А завучу нужно другое, бесплатно, по-русски, за секунды и с ответом, почему вышло именно так. Я так свою программу и сделал, таблица из Excel, кнопка, пять вариантов с разбором, правка руками и выгрузка обратно в Excel. Интернет ей не нужен, код открыл под MIT.

Цель простая, сделать бесплатный русскоязычный инструмент для двухсменной школы, который за секунды строит допустимое расписание и объясняет его [1]. Задачи у меня были такие. Первая, описать жесткие и мягкие правила. Вторая, собрать конвейер от жадного старта до независимой проверки. Третья, получить ноль окон у учеников на сложном наборе. Четвертая, дать пять непохожих вариантов с объяснением и ручную правку. Гипотеза такая. Если дать скорость, несколько непохожих вариантов и разбор словами, то выйдет не хуже ручного по окнам учителей при нуле окон у детей.

Основная часть. Возможности программы

На вход она берет классы и смены, предметы с часами, учителей, кабинеты, деления на подгруппы и правила. На выходе дает пять разных расписаний на неделю. Я специально сделал пять непохожих, похожие не вытесняют разные, проверяется по хэмминговой метрике. У каждого варианта оценка словами, «Отличное», «Хорошее» или «Требуется внимания», и разбор трех главных причин. Там конкретные имена, какой класс, какой учитель и в какой день тянет оценку вниз.

Выбранный вариант можно править руками. Каждый ход программа оценивает заранее. Либо можно, либо можно, но станет хуже, либо запрещено, и всегда пишет, почему. Это удобно, потому что завуч видит цену своего хода до того, как нажмет. Главный упор я делаю на ядро, то есть на движок, который считает. Интерфейс уже есть, он рабочий, но я буду переделывать его полностью, чтобы было понятнее и красивее. Программу делаю максимально гибкой, там много настроек. При этом обычному завучу

ничего крутить не надо, ему хватает профиля Стандарт и одной кнопки.

Запреты у меня двух видов. Жёсткие нарушать нельзя. Один учитель, один класс и один кабинет в один момент только в одном месте. Класс учится в свою смену. Половинки подгрупп начинают вместе. У учеников ноль окон и старт не позже второго урока, это строгое правило. Есть еще лимит уроков учителя в день. Мягкие это пожелания со штрафами. Окна учителей внутри смены весят 10, перерывы между сменами весят 2, они дешевле, потому что для двухсменки это нормально. Тяжелые уроки на краю дня весят 20, повторы предмета за день 15, кабинеты 5.

Поиск ограничен временем, и в программе это подписано.

Основная часть. Ядро и архитектура

Написал на C#, платформа .NET 10, окна WPF по MVVM, база SQLite без EF, на чистом Microsoft.Data.Sqlite. Решатель Google OR Tools CP SAT [3] плюс свой локальный поиск. Конвейер такой: сначала жадный старт, он собирает всю школу за доли секунды, тесные предметы вроде физкультуры и языков ставит первыми. Потом ремонт, он сдвигает дни в плотные блоки без дыр у детей. Потом локальный поиск VND. Это тысячи мелких ходов: переставить один урок или обменять два. Цену хода считает быстрый дельта индекс за $O(k)$, а не полным пересчетом, поэтому успевает. Потом страховка LNS и балансировка первых классов. В конце независимая проверка FullValidator, если хоть одно жесткое нарушение — результат отклоняется. Плохое расписание выдать нельзя физически.

Мне тут дороги три вещи. Что вариантов пять и они разные, а не копии одного. Что вместо голой цифры идет разбор словами с виновниками. И что запуск повторяется один в один, все проверяется. Быструю формулу сверял с полным пересчетом на сотнях ходов, сошлось.

CP SAT Phase B на больших школах я пропускаю осознанно. Замер показал, что на школах от 300 уроков он дает ноль улучшений за 34 до 180 секунд. На малых школах он работает, там оставил.

Основная часть. Замеры

Для проверки я взял синтетический набор, близкий к типовой двухсменной школе, но специально сложный. 40 классов, около сотни учителей, 40 кабинетов, пять дней в две смены, 1196 уроков. Плотность высокая, есть профильные пары и допы, то есть вариант «hard». Я так сделал нарочно, чтобы проверить потолок, а не красивый случай.

Этап	Штраф	Что это значит
Жадный старт	11599	0,1 секунды, все уроки расставлены
Конвейер, 18 секунд	8736	допустимо, ноль жестких
Долгая доводка	см. ниже	окна учителей от 285 до 162

Главная цифра такая. В начале доводки у учителей было 285 окон, в конце стало около 160, лучший прогон дал 162. Это минус 43%. При этом ноль окон у учеников, ноль стартов позже второго урока и ноль жестких нарушений. Обычный режим «Стандарт» на 12 секунд уже дает допустимое расписание, долгий режим просто чистит учительские окна дальше.

Разброс по разным seed узкий, то есть результат стабильный, а не везение одного запуска. Остаток это в основном окна учителей и сдвоенные предметы, я это вижу в разборе. Оптимум не заявляю, это лучшее найденное за время, так и пишу.

Надежность проверена тестами. Полный набор это 437 тестов плюс 5 пропусков, ноль падений. Пропуски объявлены заранее, это карантин одной тесной фикстуры и долгие разовые прогоны. Проверены валидатор, решатель, подгруппы, паритет быстрой формулы и полного пересчета, импорт, экспорт и весь путь от шаблона до Excel. Есть демо набор на 56 строк, на нем генерация идет за секунды, можно показать на ноутбуке за минуту.

Основная часть. Нормы и ограничения

Трудность предметов я взял из школьного сборника норм, таблица 3 из изменения Минздрава номер 35 [4]. Там баллы от 12 у математики до 3 у физкультуры. Моя шкала от 1 до 10, это те же баллы минус два, тяжелое начинается с 7. Тяжелыми вышли математика, иностранный, языки, физика и химия. Такие уроки программа уводит с краев дня и собирает в пик. Недельные нормы и пик сверял с планами и требованиями [5][6][7].

СанПиН блок в программе помечен «NEEDS-CHECK». Нормы требуют сверки с текстами, поэтому нормативную корректность я не заявляю. Это ограничение пишу и здесь, и в программе.

Заключение

Готово такое. Ядро считает, программа одним файлом, работает offline на обычном школьном ПК. Демо за минуту, выгрузка в Excel, код открыт. Интерфейс сейчас рабочий, но это временный вариант, дальше будет новый. На ближайшее будущее у меня четыре задачи. Первая, сделать более удобный и приятный интерфейс. Вторая, сверить блок СанПиН с текстами норм. Третья, опросить учителей и понять их оценку моего расписания. Четвертая, дальше улучшать качество и скорость ядра. Дальше хочу взять реальную нагрузку, прогнать целиком и сделать таблицу было и стало по времени и окнам, плюс взять отзыв школы. Ничего менять в школе не прошу, фамилии детей нигде не нужны.

Если сравнить с гипотезой, то она сошлась. Скорость, пять вариантов и разбор дали минус 43 процента по окнам учителей при нуле окон у детей. Новизна, как я ее вижу, это связка в одном месте. Скорость плюс пять непохожих вариантов плюс объяснение словами плюс бесплатно и по-русски под двухсменку в одном offline инструменте. Такого вместе я не нашел. Польза простая, завуч получает черновик за секунды и видит, что править.

Про ограничения прямо. Результат всегда лучшее найденное, не оптимальное. Таймаут это не значит невозможно, в программе есть статусы не найдено за время и остановлено, лучшее сохранено. Пар без синхрона и замен на больничный пока нет, это в планах.

Список использованных источников

1. Положение об образовательной смене НДТП, сайт ndtp.by, <https://ndtp.by>.
2. Документация конструктора расписаний FET, <https://lalescu.ro/liviu/fet/>.

3. Документация решателя Google OR Tools CP SAT, <https://developers.google.com/optimization>.

4. СанПиН номер 206 (2012) и изменение номер 35 (2017), тексты на <https://pravo.by>.

5. Типовые учебные планы Минобразования номер 47/2024, номер 75/2025, номер 104/2026, <https://adu.by>.

6. Санэпидтребования Совмина номер 525, <https://pravo.by>.

7. Письмо Минобразования на 2026/2027 учебный год, <https://edu.gov.by>.